

How Money Moves

An introduction to the accounting and mechanics of banking

Raphael Gruber

2026

Contents

Preface	1
Chapter 1 — What a Bank Is	2
Chapter 2 — Double-Entry Bookkeeping	4
Chapter 3 — The Chart of Accounts	6
Chapter 4 — Ledgers, Subledgers, and Money	10
Chapter 5 — Transactions and Postings	12
Chapter 6 — Booking Date vs. Value Date	15
Chapter 7 — Balances and Holds	17
Chapter 8 — The Account Lifecycle and Overdraft	19
Chapter 9 — Clearing and Settlement	22
Chapter 10 — The Interbank Network	24
Chapter 11 — Payment Schemes	27
Chapter 12 — SEPA: Credit Transfers and Direct Debits	30
Chapter 13 — Card Transactions	32
Chapter 14 — Snapshots, Audit Trails, and Statements	35

Preface

Most people use banks every day without ever seeing how they work on the inside. We deposit a paycheck, tap a card, send a transfer — and money seems to simply appear in one place and vanish from another. But underneath that smooth surface is a centuries-old discipline of bookkeeping, a precise vocabulary, and a set of mechanisms for moving value between institutions that almost never fail and almost never lose a cent.

This book explains that machinery from the ground up. It assumes no accounting background. It starts with the single rule that all of banking is built on — that money never appears or disappears, it only moves — and follows that rule all the way out to the interbank networks where banks settle their debts with one another through accounts at a central bank.

The examples are concrete and the numbers add up, because the ideas here are also implemented as a small, working banking library. Where it helps, the text shows how a real system models a concept. But the subject of this book is the domain, not any particular program: what banks track, why they track it that way, and how money actually moves.

What you'll learn

- Why the money in your checking account is the bank's *debt*, not its asset.
- How a 500-year-old technique — double-entry bookkeeping — makes errors detectable by construction.
- The difference between the day a transaction is *recorded* and the day it takes *economic effect*, and why banks need both.
- Why your "available balance" can differ from your "current balance," and what a hold really is.
- What actually happens, account by account, when you send money to someone at a different bank — and why "clearing" and "settlement" are not the same thing.
- How banks move millions of payments while only transferring the *net* between themselves.

Chapter 1 — What a Bank Is

Before we can talk about debits, credits, or settlement, we need a clear picture of what a bank actually *is* — not as a building with a vault, but as an economic machine. The surprising thing about banks is that almost everything they hold and almost everything they owe is just a record. The records *are* the bank.

A bank is a system of record

At its heart, a bank is the authoritative record of who owns what money. Every deposit, withdrawal, transfer, loan, and fee is an entry in that record. When people in the industry talk about a *core banking system*, they mean exactly this: the system of record for all of an institution's financial activity. It is the backbone everything else hangs from — the mobile app, the ATM network, the card rails, the regulatory reports — all of them are ultimately reading from or writing to this one source of truth.

This is why correctness matters so much. A social network can tolerate a lost "like." A bank cannot tolerate a lost cent. The entire discipline of bank accounting exists to make sure that the record is always internally consistent and that errors are caught the moment they happen rather than discovered months later.

Money, from the bank's point of view, is a promise

Here is the first idea that trips up almost everyone. When you deposit \$500 in cash at your bank, that \$500 does not "belong to you" in any account the bank keeps for you. The physical cash becomes the bank's property — it goes into the bank's vault and the bank can lend it out. What you have in return is a *claim*: the bank now owes you \$500, payable on demand.

So the balance you see in your checking account is, from the bank's perspective, a debt the bank owes you. It is a liability — an obligation. This is not a metaphor or an accounting trick; it is the literal legal and economic reality. A deposit is a loan you make to the bank.

This single reframing — *your money is the bank's promise to pay you* — unlocks most of what follows. Once you see deposits as the bank's obligations, the whole structure of bank accounting starts to make sense.

Owning and owing: the two sides of a bank

Everything a bank holds falls into one of two broad categories:

- **Things the bank owns or is owed** — its *assets*. The cash in the vault and at the central bank, the loans it has made (because borrowers owe it money), the securities it holds.
- **Things the bank owes to others** — its *liabilities*. Customer deposits, money it has borrowed from other banks, bonds it has issued.

The gap between the two — what would be left for the owners if the bank sold every asset and paid off every debt — is the *equity*, the shareholders' stake.

That relationship is the foundation of all bank accounting, and it always holds:

Assets = Liabilities + Equity

If a bank has \$100 million in assets and owes \$92 million, then \$8 million of equity belongs to its owners. This equation is never allowed to break. Every single thing a bank does — taking a deposit, making a loan, charging a fee, paying interest — is recorded in a way that keeps both sides equal. The next two chapters are entirely about *how* that balance is maintained, transaction by transaction.

A worked intuition: taking a deposit

Watch what happens to the equation when that customer deposits \$500 in cash:

- The bank's **cash** (an asset) goes up by \$500.
- The bank's **deposit owed to the customer** (a liability) goes up by \$500.

Assets		Liabilities + Equity
+\$500	=	+\$500

Both sides rose by the same amount, so the equation still balances. Notice that the bank is now *richer in cash* and *deeper in debt* at the same time — and that's completely normal. A bank growing its deposits is taking on more obligations and more assets in lockstep. The art of banking is what it does with those assets in between: lending them out at a higher rate than it pays you, earning the difference.

Why this is worth modelling carefully

It would be possible to build a banking system that just stores a single number per customer — their balance — and adds to it or subtracts from it. Plenty of toy systems do exactly that. But real banks don't, and for good reasons that the rest of this book will draw out:

- A single number can't tell you *why* a balance changed, or prove that it changed for a legitimate reason.
- A single number can't distinguish money that has been *recorded* from money that has actually *arrived* — a distinction that turns out to be central to how interest, availability, and risk work.
- A single number gives you no way to detect when something has gone wrong.

The accounting techniques in the coming chapters solve all three problems at once. They are old — the core of the method was written down in fifteenth-century Italy — but they have never been improved on, because they get something exactly right. That is where we turn next.

Chapter 2 — Double-Entry Bookkeeping

In the last chapter we met the accounting equation — assets equal liabilities plus equity — and claimed that a bank never lets it break. This chapter is about the technique that makes that promise keepable. It is the single most important idea in all of accounting, and once it clicks, everything else in this book follows from it.

One rule

The technique is called *double-entry bookkeeping*. It was codified by the Italian mathematician Luca Pacioli in 1494, drawing on methods Venetian merchants had already used for two centuries, and it has been the foundation of serious accounting ever since. The rule is almost absurdly simple:

Every transaction must have equal debits and credits.

That's it. Every time money moves, it is recorded in (at least) two places: one side records where the value came from, the other where it went. The two sides are always equal. The consequence is profound:

Money never appears or disappears. It only moves from one account to another.

There is no operation in a correct banking system that simply "creates" a balance out of nothing. Value is always conserved, because every increase somewhere is matched by a corresponding entry somewhere else.

Debits and credits

The two directions an entry can take are called **debit** and **credit**. These are the most misunderstood words in finance, mostly because everyday language has them backwards. When your bank tells you it has "credited your account," your balance went up — but that's because, as we saw, your account is the bank's *liability*, and (as we'll see) credits increase liabilities.

For now, resist the urge to attach "debit = decrease" or "credit = increase" to the words. Debit and credit are just the two sides of an entry — left and right, by long convention. Whether a debit *increases* or *decreases* a particular account depends on what kind of account it is, which is the subject of the next chapter. The only rule you need right now is the balance rule: within any transaction, the debits and the credits are equal.

Two examples

Consider a customer depositing \$100 in cash. Two things change: the bank has more cash, and the bank owes the customer more. So we record two entries:

- **Debit** the bank's Cash account — an asset increases; the bank has more cash.
- **Credit** the customer's Deposit account — a liability increases; the bank owes the customer more.

Debit \$100, credit \$100. Balanced. And notice it matches the intuition from Chapter 1: the bank got richer in cash *and* deeper in debt, both by \$100.

Now consider one customer transferring \$50 to another customer at the same bank:

- **Debit** the sender's Deposit account — the bank owes the sender less; their liability balance falls.
- **Credit** the receiver's Deposit account — the bank owes the receiver more; their liability balance rises.

Debit \$50, credit \$50. Balanced again. The bank's *total* liabilities didn't change — it still owes the same amount in total — the obligation just moved from one customer to another. No cash left the building.

Error detection for free

The reason double-entry has survived five centuries unchanged is that the balance rule is also an error-detection mechanism, built into the structure of the records themselves.

Because every individual transaction balances, the sum of *all* debits ever recorded must equal the sum of *all* credits ever recorded. If you list every account and its balance — a report called a **trial balance** — the debit balances must equal the credit balances. If they don't, you *know*, with certainty, that something is wrong: an entry was dropped, a number was fat-fingered, a transaction was only half-recorded. You may not yet know *where* the error is, but you know one exists. A system that stored only a single balance per customer could never give you that guarantee.

This is why a banking system enforces the balance rule at the moment a transaction is recorded, and refuses to record one that doesn't balance. A transaction whose debits and credits are unequal isn't a transaction with a small problem — it's a corruption of the entire record, and it is rejected outright. In the accounting core of our reference library, this is the first and most jealously guarded invariant: a set of entries is summed by direction, and if total debits do not equal total credits the posting is refused before it can touch the books.

Why "double"

It's worth pausing on the word *double*. The "double" doesn't mean exactly two entries — many real transactions have three or more, as we'll see in Chapter 5. It means every transaction is recorded from *two perspectives*: where value came from and where it went. A loan disbursement, a fee split, an interest payment — each may touch several accounts, but every one of them records both sides of the movement, and every one of them balances.

With the balance rule in hand, we have the *form* of every transaction. What we don't yet have is a way to organize the accounts those transactions touch, or to know whether a debit to a given account means "more" or "less." That requires a classification of accounts — the chart of accounts — which is where we go next.

Chapter 3 — The Chart of Accounts

We now know that every transaction balances, and that an entry is either a debit or a credit. What we're still missing is the meaning: when we debit an account, does its balance go up or down? The answer depends entirely on what *kind* of account it is. Classifying accounts into a handful of types — the *chart of accounts* — is what gives debits and credits their meaning.

Five types, from one equation

Every account a bank keeps belongs to one of five fundamental types. They are not arbitrary; they fall directly out of an expanded version of the accounting equation from Chapter 1:

$$\text{Assets} = \text{Liabilities} + \text{Equity} + (\text{Revenue} - \text{Expenses})$$

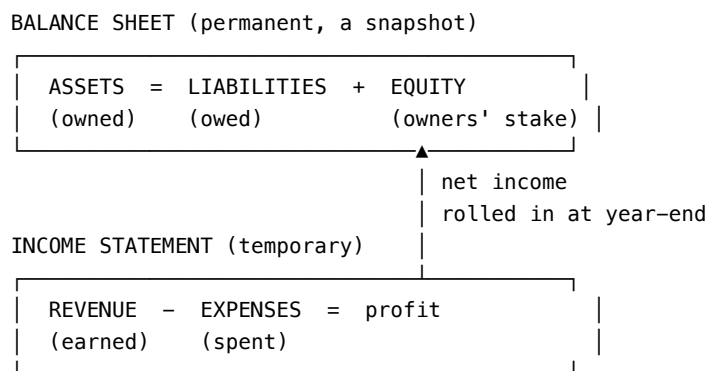
That extra term — revenue minus expenses — is the bank's profit, which belongs to the owners and so adds to equity. Pull the five nouns out of that equation and you have the five account types:

Type	What it represents	Examples
Asset	Things the bank owns or is owed	Cash, loans to customers, securities, real estate
Liability	Things the bank owes to others	Customer deposits, borrowings, bonds payable
Equity	The owners' residual interest	Paid-in capital, retained earnings
Revenue	Income the bank earns	Interest income, fee income, trading gains
Expense	Costs the bank incurs	Interest expense, salaries, rent, loan-loss provisions

These five split into two families that answer two different questions.

The three **permanent** accounts — assets, liabilities, equity — describe the bank's *position*: a snapshot of what it owns and owes at a single instant. The report that lays them out is the **balance sheet**, and it is governed by $\text{Assets} = \text{Liabilities} + \text{Equity}$. These accounts persist year after year; your deposit balance doesn't reset on January 1st.

The two **temporary** accounts — revenue and expenses — describe the bank's *activity* over a period: how much it earned and spent across a month or a year. The report is the **income statement**. At year-end these accounts are "closed": their balances are totalled into the year's profit, that profit is rolled into equity (specifically retained earnings), and they reset to zero to start counting the next period afresh. That closing step is the hinge connecting the two reports — activity over a period flows into position at a point in time.



Normal balances

Now we can give debits and credits their meaning. Each account type has a **normal balance** — the direction that *increases* it. The opposite direction decreases it.

Type	Normal balance	A debit...	A credit...
Asset	Debit	increases	decreases
Expense	Debit	increases	decreases
Liability	Credit	decreases	increases
Equity	Credit	decreases	increases
Revenue	Credit	decreases	increases

There is a neat symmetry worth memorizing: **assets and expenses are debit-normal; liabilities, equity, and revenue are credit-normal.** The two "left side of the equation" things (what you own, what you spend) increase with debits; the three "right side" things (what you owe, the owners' stake, what you earn) increase with credits. This is the whole reason the balance rule keeps the equation intact: a balanced transaction nudges both sides equally.

This table is the Rosetta Stone for everything that follows. When you read "debit Alice's deposit account," you can now decode it: a deposit account is a liability, liabilities are credit-normal, so a debit *decreases* it — Alice has less money. When you read "credit interest income," that's revenue, credit-normal, so the bank earned more.

The rest of this chapter walks each type in turn, because the intuitions behind them are where most confusion lives.

Asset — things the bank owns or is owed

An asset is anything of value the bank controls. The most intuitive example is cash in the vault — clearly something the bank owns. But assets also include money that *other people owe the bank*. This is the part people miss. When a bank gives a customer a \$200,000 mortgage, it doesn't lose \$200,000 — it *converts* one asset (cash) into another (a loan receivable). The customer now owes the bank \$200,000 plus interest, and that promise to repay is itself a valuable asset.

Typical asset accounts:

- **Cash and reserves** — physical currency and balances held at the central bank.
- **Loans to customers** — mortgages, personal loans, credit-card balances.
- **Securities** — government and corporate bonds and other investments.
- **Interbank lending** — short-term loans to other banks, such as overnight lending.

Assets are debit-normal: a debit increases an asset, a credit decreases it. When the bank receives a \$500 cash deposit, its cash increases by a \$500 debit — and, as we keep seeing, the other side of that entry is a liability.

Liability — things the bank owes

A liability is an obligation to pay someone else. For a bank, the most important liability by far is **customer deposits**. When you deposit \$500 into checking, the bank now owes you \$500 on demand. Your balance, from the bank's point of view, is a debt.

This is the single most counterintuitive fact in retail banking, so it bears repeating from Chapter 1: the money in your checking account is the bank's liability, not its asset. The bank holds the cash (an asset) but owes it back to you (a liability).

Typical liability accounts:

- **Customer deposits** — checking, savings, certificates of deposit.
- **Borrowings** — money the bank has borrowed from other banks or the central bank.
- **Bonds payable** — debt the bank has issued to raise capital.

Liabilities are credit-normal: a credit increases them, a debit decreases them. When a customer deposits \$500, the bank *credits* the deposit liability (it owes more) and *debits* cash (it holds more). Both sides increase — more cash, more debt — and the books balance.

Equity — the owners' residual interest

Equity is what's left after subtracting liabilities from assets: the shareholders' stake. A bank with \$100M in assets and \$92M in liabilities has \$8M of equity. It is a *residual* — it's defined by the other two, not measured directly.

Equity moves less often than the other accounts. It changes when the bank issues or buys back shares, and at year-end when net profit (revenue minus expenses) is rolled into retained earnings.

- **Paid-in capital** — what shareholders invested when buying the bank's stock.
- **Retained earnings** — accumulated profits kept rather than paid out as dividends.

Equity is credit-normal: profits increase it (credit), losses decrease it (debit).

Revenue — income the bank earns

Revenue accounts track money flowing in from operations. A bank's primary revenue is interest on loans: it lends at a higher rate than it pays on deposits, and the difference — the *net interest margin* — is how banks make most of their money.

- **Interest income** — earned on loans, mortgages, and securities.
- **Fee income** — account fees, ATM fees, wire fees, overdraft fees.
- **Trading gains** — profits from buying and selling securities.

Revenue is credit-normal. When a customer pays \$50 of interest on a loan, the bank *credits* interest income (it earned more) and *debits* either the loan balance (the customer repaid part of an asset) or cash (if the money came from outside the bank). At year-end, revenue is closed into retained earnings.

Expense — costs the bank incurs

Expense accounts track the cost of running the bank. Just as revenue increases the owners' stake, expenses decrease it.

- **Interest expense** — interest paid to depositors and bondholders; usually the bank's single largest cost.
- **Salaries and benefits** — employee compensation.
- **Loan-loss provisions** — money set aside against expected defaults.
- **Operating costs** — rent, technology, compliance, legal.

Expenses are debit-normal. When the bank pays a saver \$10 of monthly interest, it *debits* interest expense (a cost was incurred) and *credits* the customer's deposit (it now owes \$10 more). Like revenue, expense accounts close into retained earnings at year-end.

Putting it together

With the five types and their normal balances, you can now read any transaction in this book and predict its effect. The classification isn't bureaucracy — it's the layer of meaning that turns the mechanical balance rule of Chapter 2 into a true description of the bank's economic reality. Next we'll see how all these accounts are organized in practice, and how the amounts in them are represented so that not a single cent is ever lost to rounding.

Chapter 4 — Ledgers, Subledgers, and Money

A real bank has millions of accounts. The five types from the last chapter tell us what each account *means*, but not how to organize them, and not how to write down the amounts they hold. This short chapter closes both gaps — first the structure that keeps millions of accounts navigable, then the deceptively important question of how to represent a sum of money so that it is never wrong by a cent.

A two-level hierarchy

Accounts are organized into a two-level tree: a **ledger** at the top, divided into **subledgers**, each of which holds the individual accounts.

General Ledger

```
|— Customer Deposits (subledger)
|   |— Alice Checking      (Liability)
|   |— Bob Checking       (Liability)
|   |— ... 50,000 more accounts
|— Loans (subledger)
|   |— Loan #12345        (Asset)
|   |— ...
|— Bank Assets (subledger)
|   |— Cash Vault         (Asset)
|— Revenue (subledger)
|   |— Fee Income         (Revenue)
```

A **ledger** is the top-level book. A bank typically has one *General Ledger* (the "GL") containing all of its accounts. Very large institutions may keep separate ledgers for different legal entities or business units, but conceptually the ledger is the complete book of record for one set of accounts.

A **subledger** is a subdivision that groups related accounts: one for customer deposits, one for loans, one for interbank balances, one for fee income, and so on. The point of subledgers is summarization. The General Ledger can report a single line — "Total Customer Deposits: \$10,000,000" — while the Customer Deposits subledger underneath it holds the 50,000 individual accounts that add up to that figure. Management and regulators look at the summary; a customer service rep looks at one account in the subledger. Same data, two levels of zoom.

This hierarchy is purely organizational. It changes nothing about the accounting rules: every account still has a type and a normal balance, every transaction still balances. The tree just makes a vast number of accounts comprehensible.

Representing money: integers, not decimals

Now the question that sinks more naive financial systems than any other: how do you store an amount of money?

The tempting answer is a decimal or floating-point number — **100.50** dollars. It is also the wrong answer. Floating-point numbers cannot represent most decimal fractions exactly; **0.10 + 0.20** famously does not equal **0.30** in binary floating point. In a system that must conserve value to the cent across billions of operations, those tiny errors are unacceptable — they accumulate, they make trial balances fail to balance, and they are maddening to track down.

The industry-standard solution is simple and exact: **store every amount as an integer number of the smallest unit of the currency**. For US dollars that unit is the cent; for euros, the euro cent; for currencies with no minor unit, the whole unit. An amount is just a count of those minor units.

Display	Stored as	Unit
\$100.50	10050	cents
\$1,234.56	123456	cents
\$10,000.00	1000000	cents

This is exactly the approach Stripe, most banks, and most payment processors use. Integer arithmetic is exact: there is no rounding, no representable-fraction problem, no drift. Adding two amounts can never introduce an error, and a column of integers always sums to precisely the right total.

The cost is a small, deliberate division of responsibility: the *core* deals only in integer minor units, and converting to and from a human-friendly display string ("**\$100.50**") is the job of whatever sits at the edge — the UI, the report generator, the statement renderer. The accounting engine never sees a dollar sign and never sees a decimal point. In our reference library this is made explicit: the amount type is simply a 64-bit integer of minor units, and every balance, every entry, and every payment is counted in those units.

A 64-bit integer of cents can represent sums up to roughly 92 quadrillion dollars before overflowing — comfortably more than any real balance — while keeping every single cent exact. For the rare cases that need fractional cents (some interest accruals, some FX), systems extend the same idea by choosing an even smaller unit (say, millionths), never by reaching for floating point.

Why these two choices travel together

The hierarchy and the integer representation are both in service of the same goal: a record that is *exactly* correct and *easy to reason about* at any scale. The subledger structure means you can always drill from a summary down to the individual entries that justify it. The integer amounts mean that when you do, the numbers will add up to the penny. Together they make the next idea possible — the idea that the record is so trustworthy you never edit it, you only ever append to it. That brings us to transactions themselves.

Chapter 5 — Transactions and Postings

We have the balance rule, the five account types, and an exact way to write down money. Now we can talk about the unit of work that ties them together: the *transaction*. This chapter pins down the vocabulary that surrounds it — words that practitioners use loosely and interchangeably — and then covers the three properties that make a bank's transactions trustworthy: they can be larger than two sides, they are safe to retry, and they are never erased.

A precise vocabulary

A handful of words get used almost interchangeably in banking, which is fine once you know they're synonyms but bewildering until then. Here is the precise set.

- **Entry** — a single debit or credit to one account. It carries an account, an amount, and a direction (debit or credit). It is the smallest unit of bookkeeping; everything else is built from entries.
- **Leg** — a synonym for *entry*. The word is used when emphasizing that a transaction has several sides: a "two-legged" transfer has one debit leg and one credit leg; a "multi-legged" transaction has three or more. A leg *is* an entry; there is no separate concept.
- **Posting** — the act of recording a transaction in the ledger (the verb: "to post a transaction"). Loosely, "a posting" can also mean an entry that has been recorded. The key fact about posted transactions is that they are *final*: they are never edited or deleted, only reversed (more on that below).
- **Transaction** — a balanced set of entries that are posted together as one atomic unit. "Atomic" means all-or-nothing: either every leg is recorded or none is. And within any transaction, **total debits equal total credits** — the balance rule from Chapter 2, now stated as a property of the transaction as a whole.

One more term you'll meet throughout the book: a **trial balance** is the list of every account's balance at a moment in time. Because every transaction balances, the debit balances across all accounts must equal the credit balances. A trial balance that doesn't sum to zero is a proof that a bookkeeping error exists somewhere — the self-checking property from Chapter 2, applied to the whole book.

Transactions can have many legs

The textbook example of double-entry is a two-legged transfer: debit one account, credit another, equal amounts. But real transactions frequently need more than two legs, and the balance rule handles them with no modification — the *total* debits must equal the *total* credits, however many legs there are.

A few common multi-legged shapes:

- **A fee split.** A customer sends \$100, but \$3 of it is a transfer fee. One debit of \$100 from the sender; one credit of \$97 to the recipient; one credit of \$3 to the bank's fee-income account. Debits: 100. Credits: $97 + 3 = 100$. Balanced.
- **A loan disbursement.** Paying out a \$10,000 loan might credit the customer's deposit account, debit a loan-receivable asset, and — if there's an origination fee — debit the deposit account for the fee with a matching credit to fee revenue. Several legs, still balanced.
- **Interest accrual with tax.** Posting monthly interest might debit the bank's interest-expense account and credit the customer's deposit, with a third leg withholding tax into a liability account owed to the tax authority.

The invariant is the same every time: **total debits = total credits**. A system enforces it by summing each direction at the moment of posting and refusing anything that doesn't match — which is exactly what our reference library does before it will write a transaction to the books. The number of legs is irrelevant; the balance is not.

Idempotency: making retries safe

Banks run on networks, and networks fail in the worst possible way: not by losing a request outright, but by losing the *response*. A client sends "transfer \$100," the transfer is recorded, and then the acknowledgement is lost on the way back. The client, having heard nothing, retries. Without a defense, the bank posts the transfer a second time and the customer is debited twice.

The defense is an **idempotency key**. The client attaches a unique identifier — a UUID, say — to each logical operation. The rule is:

1. The client generates a unique key for each logical operation and includes it in the request.
2. If the system sees a key it has already processed, it refuses to create a duplicate and instead reports that the key was already used.
3. The client can then look up the original transaction by that key and learn what happened, rather than guessing.

The key turns an unreliable network into a safe one. A retry with the same key is recognized as the *same* operation, not a new one, so retrying becomes harmless — exactly the property you want when you can never be sure whether your first attempt got through.

Immutability and reversal

Here is a rule that surprises engineers coming from ordinary databases: in banking, **posted transactions are never deleted, and never edited**. The ledger is an immutable, append-only record. This isn't a limitation to work around; it's a feature that underpins the whole system's trustworthiness. An auditor, a regulator, or a forensic investigator must be able to see exactly what was recorded and when, with no possibility that history was quietly rewritten.

So how do you fix a mistake? You don't reach back and change the bad entry. You post a *new* transaction — a **reversal** — that exactly cancels the original:

- Every debit in the original becomes a credit in the reversal.
- Every credit in the original becomes a debit in the reversal.
- The net effect across all affected accounts is zero — it's as if the original never happened, economically, while remaining fully visible historically.

The original transaction is marked as "reversed" for reporting, and the reversal carries a reference back to the transaction it undoes, so the relationship between the mistake and its correction is permanent and auditable. Two true facts now live in the record side by side: that the original posting was made, and that it was later unwound. Nothing was hidden; the correction is itself part of the history.

This append-only discipline is the same instinct behind the integer amounts of Chapter 4 and the trial balance of Chapter 2 — build the record so that it is *provably* correct and *provably* complete, and never give yourself the ability to quietly break that promise.

What we still can't express

Everything in this chapter has treated a transaction as a single event at a single moment. But banking runs on a subtler notion of time. The day a transaction is *recorded* is often not the day it takes *economic effect* — and that gap, far from being an edge case, drives interest, availability, and risk. That is the subject of the next chapter.

Chapter 6 — Booking Date vs. Value Date

Ask someone when a transaction "happened" and they'll give you one date. Banking needs two. The gap between the moment a transaction is *recorded* and the moment it takes *economic effect* is not a technicality — it is woven through interest calculations, fund availability, regulatory reports, and risk. This chapter is about those two clocks and why a bank must keep both.

Two dates on every transaction

Every transaction carries two dates:

- **Booking date** — when the transaction was recorded in the system. This is the "system date" or "processing date." It determines when the transaction shows up in audit trails and operational reports. It answers: *when did we write this down?*
- **Value date** — when the transaction takes economic effect. This determines when interest starts or stops accruing, when funds become available, and which business day "owns" the money. It answers: *when does this money actually count?*

For a simple cash deposit handed to a teller, the two coincide — recorded and effective at the same instant. But in a great many real situations they diverge, sometimes by days or weeks, and the value date can fall either *after* the booking date (the common case) or even *before* it.

When the two dates diverge

A handful of everyday scenarios show why the second date is unavoidable:

- **Weekend and holiday processing.** A wire that arrives Friday evening is booked immediately — booking date Friday, 7:00 PM — but the funds only take effect on the next business day, so the value date is Monday. Across a long holiday weekend the gap can stretch to four or five days.
- **Check deposits.** A customer deposits a check on Monday and the bank records it at once (booking date Monday). But the check has to clear through the interbank system, so its value date might be Wednesday or Thursday. Until the value date arrives, the money earns no interest and may not be available to spend.
- **Back-dated corrections.** On March 5, an operations team realizes a corporate payment should have taken effect on February 28. They book the correction today — booking date March 5 — but give it a value date of February 28, so that interest for the intervening days is computed correctly. Here the value date is *earlier* than the booking date. Without this, the customer would lose several days of interest.

- **Forward-dated standing orders.** A customer schedules rent for the 1st of next month. The bank may record the instruction today (booking date January 20) but assign a value date of February 1, when the money actually moves. Here the value date is in the *future*.
- **Securities settlement.** A stock trade executed Monday (trade date, "T") typically settles the next business day ("T+1"). The cash leg is booked Monday but value-dated to Tuesday, when ownership and funds actually change hands.

The unifying principle is this: **interest, availability, and reporting follow the value date, not the booking date.** Use the wrong one and customers earn too much or too little interest, and regulatory balance reports come out wrong. A check deposited Friday does not earn a weekend's interest if its value date is Monday — and getting that right requires the system to remember both dates for every transaction.

Who decides the value date

The value date isn't chosen by any single actor; it depends on the type of transaction, and several parties have a hand in it:

- **Automated rules** handle the vast majority. The bank's systems assign a value date from predefined policies per product and channel — domestic wires get same-day value, checks get T+2, international transfers get T+1 to T+3 depending on the corridor. In practice a rules engine sits upstream of the ledger and stamps the value date before the transaction is ever posted.
- **Payment networks** can dictate it. A SWIFT message for an international transfer carries a value-date field set by the sending bank, which the receiving bank is expected to honor. Securities markets impose conventions like T+1 that both sides agree to in advance.
- **The customer** influences it for scheduled payments and standing orders — they choose when a payment should take effect, and that becomes its value date.
- **Operations staff** set it by hand for corrections and adjustments, choosing the date when the economic event actually occurred or should have.
- **Regulation** bounds all of the above. Rules such as the US Expedited Funds Availability Act (Regulation CC) cap how long a bank may hold check funds, putting a legal ceiling on how far the value date can lag the booking date.

The takeaway for anyone building or reasoning about a banking system: by the time a transaction reaches the ledger, its value date has usually already been decided by a layer of policy above it. The ledger's job is to faithfully record both dates; the *choice* of value date is a business decision made upstream.

A glimpse ahead: the same gap, named differently

This two-date distinction reappears under other names later in the book. When we look at balances in the next chapter, we'll find that an account has *different balances depending on which date you compute it from*. When we reach clearing and settlement in Part IV, we'll see that the booking-vs-value gap is precisely the window between *initiating* a payment and *settling* it — the span during which the money has been promised but has not yet, finally, moved. The two clocks introduced here are the same two clocks that govern how money moves between banks; we're just meeting them first in their simplest form.

Chapter 7 — Balances and Holds

"What's my balance?" sounds like a question with one answer. It isn't. A single account has several balances at the same moment, each computed differently and each used for a different purpose. Understanding why is the key to a string of everyday mysteries: why your available funds differ from your statement balance, why a gas station "holds" \$100 when you bought \$40 of fuel, and why the money you just deposited isn't spendable yet. This chapter untangles the balances, then explains the mechanism — the *hold* — that sits behind most of the confusion.

One account, three balances

At any instant, an account carries three distinct balances:

- **Book balance** (also called the *ledger balance*). The balance computed from every posted transaction, by booking date, regardless of value date. It reflects everything that has been recorded. This is the figure most people think of as "the balance."
- **Value-date balance** (also called the *interest-bearing balance*). The balance computed only from transactions whose value date has already passed — the economic reality of the account. This is what the bank uses to calculate interest, generate end-of-day snapshots, and produce regulatory reports. It is the balance that "really counts" today.
- **Available balance**. The book balance minus anything currently reserved by holds (and adjusted for any overdraft limit, which we'll meet in the next chapter). This is what an ATM or a point-of-sale terminal checks to decide whether to approve a transaction — the money you can actually spend right now.

These three can all differ at once. A single account might show:

Balance	Amount	What drives it
Value-date balance	\$9,500	only transactions whose value date has passed
Book balance	\$10,000	all posted transactions
Available balance	\$9,200	book balance minus \$800 of active holds

The value-date balance sits *below* the book balance here because a forward-dated transaction has been booked but hasn't reached its value date yet — recorded, but not yet economically real. (It can also sit *above* the book balance, if a back-dated correction added value to a past date.) The available balance sits below both because \$800 is currently reserved. Each number is correct; they just answer different questions. This is the booking-vs-value distinction from Chapter 6, showing up as a difference you can read right off the account.

Holds: reserving money before the amount is known

The available balance differs from the book balance because of **holds**, and holds exist to solve a specific real-world problem: sometimes you must *reserve* funds before you know the final amount.

The classic case is paying at the pump. The sequence runs:

1. **Authorization.** You swipe your card at a gas station. The bank doesn't yet know how much fuel you'll buy, so it places a hold — say \$100 — on your account. Your *book balance* is unchanged; no money has moved. But your *available balance* drops by \$100, so you can't simultaneously spend that money elsewhere.
2. **Capture.** You finish pumping \$45 of fuel. The hold is *captured* for the actual amount: the \$100 reservation is removed and a real \$45 transaction is posted to the ledger.
3. **Release.** If the sale is cancelled instead — you drive off without pumping — the hold is simply *released*. The reservation disappears and your available balance is restored, with no trace in the ledger.

This same authorization–capture pattern underlies card payments generally, hotel and rental-car deposits, and any flow where funds must be earmarked before the final figure is settled.

Holds live off the ledger

Here is the subtle and important part. An active hold **is not a ledger transaction**. The ledger, as we've insisted since Chapter 2, records only posted transactions — actual debits and credits that have settled and that balance. A hold moves no money. It is an *operational* reservation, tracked alongside the ledger, not within it. It never appears in the general ledger and never disturbs the trial balance.

A hold touches the ledger at exactly one moment: when it is **captured**. At that point — and only then — a real, balanced transaction is posted for the actual amount: the customer's deposit liability is debited and the counterparty credited, just like any other transaction from Chapter 5. If the hold is **released** instead, *nothing ever reaches the ledger*; from an accounting standpoint it is as though it never happened.

This separation is deliberate and clarifying. The ledger stays a pure record of *settled value* — every entry in it is real, final money that moved. The operational state of money that is merely *promised or reserved* lives one layer up. In our reference library this is an explicit architectural seam: the pure ledger core knows only posted transactions and book balances, while a deposit layer on top of it owns account status, holds, and the available balance they reduce. The ledger only gets involved when a hold is captured and becomes a real transaction. Keeping the two apart is what lets the ledger remain provably consistent while the messier, more operational world of authorizations does its work without ever corrupting the books.

The shape of the available balance

We can now write the available balance as a formula. Ignoring overdraft for the moment (the next chapter adds it), it is:

Available balance = Book balance – Active holds

Holds also typically carry an expiration. If a hold isn't captured within its window, it stops affecting the available balance automatically — which is why a pending charge you never completed eventually “falls off” and your spendable funds return on their own.

With balances and holds in hand, we've covered how money behaves within a single account at a single point in time. The next chapter zooms out to the account's whole life — the states it passes through from opening to closure — and to what happens when someone tries to spend money that isn't there.

Chapter 8 — The Account Lifecycle and Overdraft

So far an account has been a static thing: it exists, money flows through it, balances are computed. But a real account has a *life*. It is opened, used, perhaps forgotten, perhaps frozen by a court, and eventually closed — and at each stage, different operations are permitted. This chapter follows that lifecycle, then turns to the question every account eventually raises: what happens when someone tries to take out more than is there?

Accounts have states

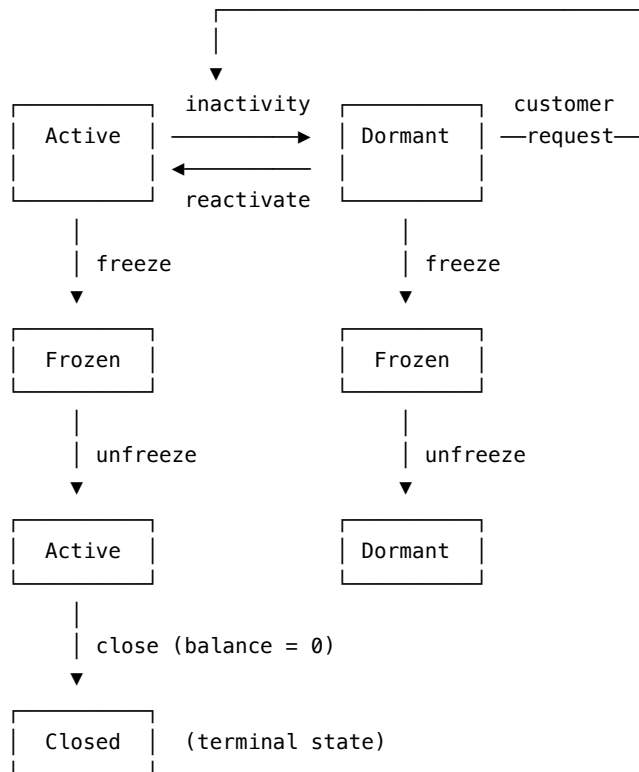
A bank account isn't simply created and then usable forever. It moves through a small set of well-defined states, and the state governs what you're allowed to do with it. There are four:

State	What it means
Active	Normal operating state; open and fully functional.
Dormant	No customer activity for a long period (often 12–24 months, varying by jurisdiction). The bank may charge dormancy fees.
Frozen	Temporarily restricted, usually by court order, fraud investigation, or regulatory action.
Closed	Permanently shut. Accepts no transactions.

These states exist because banking is a regulated activity with obligations beyond the customer's wishes. Dormancy rules protect forgotten money (and eventually route it to the state as unclaimed property). Freezes let the bank comply with law enforcement and protect victims of fraud. Closure has to guarantee that no value is stranded. The state machine encodes all of that as enforceable rules rather than leaving it to ad-hoc checks.

How accounts move between states

The transitions between states are themselves governed by rules:



The rules that matter most:

- **Active** → **Dormant** happens automatically after a configurable period of inactivity. The bank usually must notify the customer before the switch.
- **Dormant** → **Active** is triggered by any customer-initiated activity, or by an explicit reactivation request. This is why dormant accounts still accept credits: an incoming payment is exactly the kind of activity that should wake the account up.
- **Any** → **Frozen** can happen at any moment for legal or fraud reasons. Crucially, the *previous* state is remembered, so that when the freeze is lifted the account returns to where it was — an account frozen while dormant goes back to dormant, not to active.
- **Active** → **Closed** is permitted only when the balance is exactly zero, with all holds resolved and all scheduled payments cancelled. You cannot close an account that still owes or is owed money; the value has to go somewhere first.
- **Closed is terminal.** A closed account is never reopened. A customer who wants to bank again gets a brand-new account. This permanence is what makes "closed" a reliable statement for audits and regulators.

A system models this as a true state machine, with each transition checking the current state and the relevant precondition (a zero balance to close, a remembered prior state to unfreeze) before it's allowed. Like the immutability of the ledger in Chapter 5, the value is in making the rules *enforced by construction* rather than hoped for.

Overdraft: spending money that isn't there

An **overdraft** occurs when a transaction would push an account's available balance below zero. There's more than one way a bank can respond, and the choice is a business policy, not an accounting one:

- **Hard decline.** The transaction is simply rejected. This is the strictest model — if a debit would push the available balance negative, the system refuses it and returns an “insufficient funds” error. It is what our reference library does for the account types where going negative makes no economic sense.
- **Arranged overdraft (a credit facility).** The customer has a pre-agreed overdraft limit, and transactions are allowed as long as the negative balance stays within it. An account at \$0 with a \$500 limit can be debited up to \$500. Interest is charged on the overdrawn amount, usually at a higher rate than normal lending — because an overdraft *is* a short-term loan.
- **Unarranged overdraft.** The bank may, as a courtesy, honor a transaction that exceeds the arranged limit (or where none exists), but typically at much higher fees. Many jurisdictions cap those fees by regulation.

From a pure ledger standpoint, an overdraft is nothing special. The book balance of a liability deposit account simply goes negative — and a negative liability means the relationship has flipped: the customer now owes the bank rather than the bank owing the customer. The accounting just records this economic reality. The *overdraft limit* is a business rule enforced **before** posting; the ledger itself doesn't police limits, it faithfully records whatever results.

A worked example makes the interaction with the available balance concrete:

Account balance:	\$200	(bank owes the customer \$200)
Overdraft limit:	\$500	
Transaction:	-\$600	(a debit of \$600)
New balance:	-\$400	(the customer now owes the bank \$400)
Available for further:	\$100	(limit \$500, of which \$400 is used)

Folding overdraft into the formula from the previous chapter, the available balance becomes:

Available balance = Book balance + Overdraft limit - Active holds

The overdraft limit *raises* the floor of what's spendable; active holds *lower* it. Between the lifecycle states that gate which operations are even permitted, and the overdraft policy that decides how far a balance may fall, an account's behavior is fully specified — within a single bank. The next part of the book leaves the single bank behind and asks the harder question: how does money move *between* banks at all?

Chapter 9 — Clearing and Settlement

Everything so far has happened inside a single bank's books. But the payment you care about most — sending money to someone at a *different* bank — crosses a boundary that the previous chapters never had to. The money has to leave one institution's ledger and arrive in another's, and the two banks do not, and must not, write in each other's books. Bridging that gap is the job of *clearing* and *settlement*, and the distinction between them is one of the most important — and most misunderstood — ideas in banking.

Executing is not the same as settling

When Alice sends Bob \$100 by bank transfer, it feels instantaneous. Underneath, three distinct things happen, usually at three different times:

1. **Initiation.** Alice's bank debits her account and creates a payment instruction — a message that says "move \$100 from Alice to Bob at the other bank."
2. **Clearing.** The payment networks validate, match, and route that instruction alongside everyone else's. The banks work out who owes whom, and by how much. No money has actually moved between the banks yet.
3. **Settlement.** The actual funds move between the banks. Only now is the payment *final* and irrevocable.

The crucial separation is between *executing* a payment — recording the intent, debiting Alice — and *settling* it — the real, final transfer of value between institutions. The span between initiation and settlement is the **settlement cycle**, and during it the payment exists in a kind of limbo: promised, recorded, but not yet final.

This is the same booking-vs-value gap from Chapter 6, now playing out between institutions. The booking date is when the bank recorded the instruction; the value date is when settlement actually occurs and the funds become economically real.

Settlement cycles: T-plus-something

How long is that limbo? It depends entirely on the payment type. The convention for describing it is "T+n," where T is the trade or transaction day and n is the number of business days until settlement:

Payment type	Typical cycle	Notes
Card payments	T+1 to T+2	The merchant's bank receives funds 1–2 business days later. The cardholder's bank settles with the merchant's bank.
ACH / direct debit	T+1 to T+2	Batched and settled through an automated clearing house. Same-day options exist for certain payments.
Domestic wire transfers	T+0 (same day)	Settled in real time through systems like Fedwire (US) or CHAPS (UK).
International wires	T+1 to T+3	Routed through correspondent banks over SWIFT; each intermediary adds a day.
Check deposits	T+1 to T+5	Highly variable; regulation (e.g. Reg CC in the US) caps the hold. Bank of America has a 2-day hold for new customers.
Securities (stocks, bonds)	T+1	Equity markets ran on T+2 for years, then moved to T+1 in 2024.
Real-time payments	Instant	Systems like FedNow (US), Faster Payments (UK), and SEPA Instant (EU).

Payment type	Typical cycle	Notes
--------------	---------------	-------

Notice the range: from *instant* to nearly a week. The cards example shows why the holds of Chapter 7 exist — the immediate hold bridges the days until settlement, so the cardholder can't spend money that's committed but not yet moved.

What the gap costs, and how banks manage it

The settlement window isn't free. The fact that money is promised but not yet final creates real consequences that the bank has to manage:

- **Interest follows the value date.** Accrual starts when settlement happens, not when the instruction was booked. A check deposited Friday earns no weekend interest if its value date is Monday. (Chapter 6, again.)
- **Counterparty risk lives in the window.** If the *sending* bank fails between initiation and settlement, the funds may never arrive. This is the deep reason settlement *finality* matters: it marks the precise instant after which the payment can no longer be unwound and the receiving side is truly safe. Everything before finality is, in principle, reversible.
- **Holds bridge the gap for customers.** As above, a deposit hold lets the bank record incoming funds without letting the customer spend them until settlement is confirmed — protecting the bank if the payment ultimately fails.
- **Reconciliation closes the loop.** Banks hold accounts *with each other* (called *nostro* and *vostro* accounts — Latin for "ours" and "yours"). After settlement, each side checks that what it expected to receive matches what actually arrived, and any discrepancy is investigated. We'll see these mirror-image accounts in detail in the next chapter.

Why this needs more than one ledger

Here is the conceptual problem the rest of Part IV solves. A single ledger, of the kind we've built up over the last eight chapters, is perfect for recording value *within* one bank. But a cross-bank payment is precisely a movement *between* two banks' separate ledgers — and neither bank can be allowed to reach into the other's books and conjure a credit. So where does the money "exist" while it's in flight? Who holds the authoritative record during the settlement window? And how do banks that don't trust each other's ledgers agree on what moved?

The answer is a shared, trusted third party — a central bank — where every bank holds reserves, and where settlement actually happens. That structure, and the elegant way it makes "clearing" and "settlement" concrete rather than abstract, is the subject of the next chapter.

Chapter 10 — The Interbank Network

The last chapter left us with a puzzle: a cross-bank payment has to move value between two ledgers, but no bank may write in another's books. The solution is one of the quiet marvels of the financial system — a shared meeting point where banks hold reserves and settle with one another. This chapter builds that structure piece by piece and shows how it makes the abstract distinction between clearing and settlement into something concrete you can follow account by account.

Each bank keeps its own book; they meet at the center

The structural choice that makes everything work is this: **every bank keeps its own ledger**, complete and private, holding its own customers' accounts. And there is **one additional ledger for a central bank**. Banks never write into one another's books. They meet in exactly one place — the central bank — where each holds a **reserve account**.

That single design decision is what turns "clearing" and "settlement" from vague words into distinct, observable events:

- **Clearing** is the exchange and *netting* of payment instructions. The banks compare what they owe each other and agree on the net figures. No central-bank money moves during clearing — it is pure bookkeeping about who owes whom.
- **Settlement** is the movement of *reserves* between banks at the central bank. This is the moment of *finality* from Chapter 9 — when value actually, irrevocably changes hands.

Because the only place banks' money actually moves is the central bank, settlement has a single, unambiguous location and moment. Everything before it is just agreement; settlement is the act.

The accounts each bank keeps

To participate, a bank's chart of accounts (Chapter 3) needs three accounts beyond its ordinary customer deposits:

Account	Type	Purpose
Customer deposits	Liability	What the bank owes each customer (Chapters 1–3).
Clearing suspense	Liability	A holding account for in-transit funds that have left a customer but not yet settled.
Reserve at central bank	Asset	The bank's claim on the central bank. It moves only at settlement, and it <i>mirrors</i>

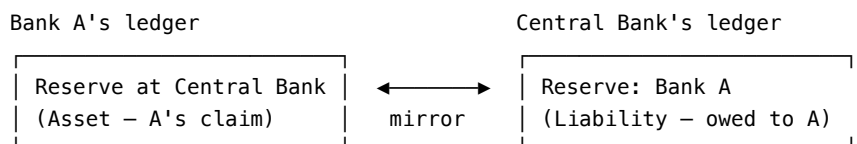
The **clearing suspense** account is the bookkeeping home for money in flight. When a customer's money leaves their account at initiation but hasn't yet reached the other bank, it can't simply vanish — double-entry forbids that. It rests in clearing suspense, which is precisely the ledger's representation of "promised but not yet settled." Once the cycle settles, suspense empties back to zero.

The **reserve at central bank** is the bank's own record of the reserves it holds centrally. Here we meet the *nostro/vostro* relationship from Chapter 9 made concrete: this asset on the bank's books is the mirror image of a corresponding liability on the central bank's books.

The central bank's book

The central bank keeps a ledger too, and from its point of view the relationship is inverted. For each participating bank it holds a **reserve liability** — because the reserves a bank parks at the central bank are money the *central bank owes that bank*. (Symmetry with Chapter 1: a deposit is always a liability of the institution holding it, and a bank's reserves are, in effect, its deposit at the central bank.) It also keeps a balancing account used when reserves are first funded.

So the same pile of reserves appears twice, in mirror image:



When these two move in lockstep — A's asset rising exactly as the central bank's "owed to A" liability rises — the books *reconcile*. Reconciliation, which sounded like an operational chore in Chapter 9, is really just the requirement that these mirror-image entries always agree. If they ever diverged, someone would have a record of money that the other side doesn't, and that is exactly the error double-entry exists to make impossible.

Following a payment through the structure

We can now narrate a cross-bank payment in terms of these accounts, leaving the exact debits and credits for the next chapter. Alice (Bank A) pays Bob (Bank B):

1. **Initiation, in Bank A's book only.** Alice's deposit is debited; the money lands in Bank A's clearing suspense. Alice's balance has fallen, but no central-bank money has moved and Bob has nothing yet. Bank A now owes "the network" the amount sitting in suspense.
2. **Clearing.** At a cut-off time, the network tots up all the instructions exchanged in this cycle and computes each bank's *net* position. Still no money moves. (Why net rather than gross is the heart of the next chapter.)
3. **Settlement, at the central bank.** Reserves move between the banks' reserve accounts at the central bank — A's down, B's up — and each bank mirrors that move in its own "reserve at central bank" asset. Bank B releases the funds from *its* clearing suspense into Bob's deposit account. Afterwards, both banks' suspense accounts are back to zero, and each bank's reserve asset again equals the central bank's matching liability. The books reconcile, and the payment is final.

Notice what this structure achieves. At no point did either bank touch the other's ledger. Bank A only ever wrote in Bank A's book; Bank B only in Bank B's; the central bank only in its own. They coordinated entirely through reserves at a shared third party they all trust. That is the whole trick of interbank money movement — and it scales from two banks to thousands without changing shape.

Why a separate layer

It's worth stepping back to see why this is modeled as something *above* the single-bank ledger rather than baked into it. A ledger answers "what does this bank own and owe?" A payment system answers "how does money move *between* banks?" They are different questions, and real institutions keep them separate: the payment rails — SEPA, the card networks, the central bank's real-time settlement system — sit *above*

each bank's accounting system and send it instructions. The ledger has no idea payments exist; it just faithfully records the postings it's told to make. Our reference library mirrors this exactly: a payment network orchestrates postings into each participant's independent ledger, while the ledgers themselves remain blissfully unaware of the network above them.

With the structure in place, one question remains: why do banks settle the *net* rather than every payment one by one, and what do the actual postings look like? That is the subject of the final chapter of Part IV.

Chapter 11 — Payment Schemes

We now have the stage — independent bank ledgers meeting at a central bank — and the cast of accounts that money moves through. This final chapter of Part IV brings on the actual choreography. We'll see why banks settle the *net* of their payments rather than each one individually, follow a real payment posting by posting, and then meet the abstract *scheme model* — the small set of axes along which any payment product differs — that the named schemes in the chapters to follow all share. SEPA's credit transfer and direct debit come first, in Chapter 12; card transactions get their own treatment in Chapter 13.

Netting: the whole point of clearing

Start with the question the last chapter deferred: why net? Imagine a single clearing cycle with just two payments between two banks:

- Alice (Bank A) → Bob (Bank B): **\$300**
- Bob (Bank B) → Alice (Bank A): **\$100**

The *customers* must move by the full, gross amounts — Alice is \$300 poorer and \$100 richer, Bob the reverse. Their deposit accounts reflect every cent of both payments. But the *banks* don't need to shuffle reserves twice. They compute each bank's net position:

```
net[A] = -300 + 100 = -200    // Bank A owes 200, net
net[B] = +300 - 100 = +200    // Bank B is owed 200, net
```

Only **\$200** of central-bank reserves actually moves, not \$400. Across a real cycle with millions of payments, this collapses an enormous gross flow into a tiny net one — vastly less reserve money has to change hands, which is faster, cheaper, and less risky. This is precisely why clearing exists as a separate step before settlement: clearing is where the netting happens.

There's also a beautiful self-checking property here. The net positions of all banks in a cycle *always sum to zero* — every dollar one bank owes is a dollar another is owed. That is exactly what guarantees the central-bank settlement transaction will balance in the double-entry sense of Chapter 2. Netting and the balance rule are two views of the same conservation law.

Following the postings: a credit transfer

Let's make a single payment fully concrete. Alice at Bank A pays Bob at Bank B €30.00 — **3000** in minor units (Chapter 4). Watch the accounts from Chapter 10 move.

1. Initiation — recorded in Bank A's ledger only, value-dated to the settlement date:

```
Bank A: Debit Alice (Liability)           3000    // Alice's deposit falls
        Credit Clearing Suspense (Liability) 3000    // Bank A now owes the network
```

Alice's money has left her account, but it hasn't reached Bob — it's parked in Bank A's clearing suspense, the "in flight" account. The transaction balances: 3000 debit, 3000 credit.

2. Clearing — at the cut-off, net positions are computed. With just this payment, $\text{net}[A] = -3000$ and $\text{net}[B] = +3000$. No money moves; this is pure agreement.

3. Settlement — three balanced transactions make the money final:

```
Central Bank:  Debit  Reserve: Bank A (Liability) 3000    // A's reserves fall
                Credit Reserve: Bank B (Liability) 3000    // B's reserves rise

Bank A:        Debit  Clearing Suspense 3000              // suspense clears to zero
                Credit Reserve at CB    3000              // A's reserve asset falls in step

Bank B:        Debit  Clearing Suspense 3000              // release the funds...
                Credit Bob (Liability)  3000              // ...into Bob's account
                Debit  Reserve at CB    3000              // B's reserve asset rises
                Credit Clearing Suspense 3000              // and its suspense clears
```

When the dust settles, both banks' clearing-suspense accounts are back to zero — nothing is "in flight" any more — and each bank's *Reserve at Central Bank* asset again equals the central bank's matching *Reserve: <Bank>* liability. The mirror-image entries agree; the books reconcile (Chapter 10); the payment is final. Trace any single account and you'll find its debits and credits balance, and trace the whole system and you'll find no value was created or destroyed — it simply moved from Alice to Bob by way of reserves at the central bank.

Schemes: different products, shared machinery

"Bank transfer" is not one thing. A salary credit, a utility's monthly collection, an instant peer-to-peer payment, and a card purchase are all "payments," but they behave differently: who starts them, whether the payer pre-authorized them, how fast they settle, whether they can be clawed back. Banking captures these differences as distinct **payment schemes**, each with its own rules.

What's elegant is that the schemes vary along just a few axes, and underneath they share the same posting machinery we just traced. The axes that matter:

- **Direction** — **push or pull**. In a *push* payment, the payer's bank initiates and sends the funds. In a *pull* payment, the payee's bank initiates and *collects* from the payer. This governs who starts the flow and whether prior authorization is needed — but, crucially, money always ends up flowing **debtor** → **creditor** regardless of who pushed the button. That's why the same postings serve both.
- **Settlement model** — **net or gross**. A *net* scheme batches payments into clearing cycles and settles the net, as we just saw. A *gross* scheme settles each payment individually and immediately, with no netting and no waiting for a cut-off.
- **Mandate required?** Some schemes require the payer to have signed a standing authorization (a *mandate*) before the payee may pull funds.
- **Returns allowed?** Some schemes let a settled payment be clawed back within a window.
- **Settlement delay** — how long after booking the funds take economic effect, i.e. the value date (Chapter 6) the postings carry.

In our reference library, these axes are literally an interface that each scheme implements; the network orchestrator drives any scheme without knowing its specifics, and adding a new product means describing it along these axes rather than rewriting the engine. The named schemes that follow — SEPA in Chapter 12, card flows in Chapter 13 — each slot into this model rather than standing apart from it.

What's deliberately left out, and what comes next

It's worth being explicit that a *learning* model of all this makes deliberate simplifications — the kind a production processor cannot. The reference library, for instance, uses one currency and no foreign exchange; routes payments by simple identifiers rather than validating real IBANs and BICs; stands in a plain payment record for ISO 20022 messages; settles returns immediately rather than in a later batch; and serializes whole operations under a lock instead of using the locked settlement windows or two-phase commit a real real-time gross settlement system would demand. None of these change the *concepts* in this chapter; they just keep the model small enough to read in an afternoon.

One further scheme shows where the same foundation naturally extends:

- **Instant payments** (SEPA Instant, FedNow, Faster Payments) are real-time *gross* settlement, 24/7. Each payment settles individually and immediately rather than waiting for a clearing cycle — the “gross” settlement model named above. The abstraction already anticipates this; what it needs is a settlement path that, for a gross scheme, posts the debtor leg, the central-bank reserve move, and the creditor leg in one shot per payment, with no netting and no cut-off.

Card transactions get their own chapter next (Chapter 13).

Instant payments and card transactions alike motivate enforcing the account *states* of Chapter 8 on the debit path — an account in a restricted state should block payment initiation — and checking that a bank actually holds enough reserves before its net settlement is allowed to post. The themes of the whole book, in other words, converge here: the balance rule, the two clocks, holds, account states, and immutable correction all reappear in the machinery that moves money between banks.

With that, the scheme model is in place. Chapter 12 puts it to work on the two SEPA products — the credit transfer push and the direct debit pull — and Chapter 13 extends it to card flows.

Chapter 12 — SEPA: Credit Transfers and Direct Debits

Chapter 11 gave us the shared machinery — clearing suspense, central-bank reserves, and the netting cycle — and the axes that distinguish one scheme from another: push or pull, net or gross, mandate required, returns allowed. Now we meet the two net-settled SEPA schemes by name. The worked example in Chapter 11 *was* a SEPA credit transfer; the postings there are the postings here, and there is no need to repeat them. This chapter layers on the scheme-specific rules: who may initiate, what authorization is required, and what happens when a settled payment needs to be unwound.

SEPA Credit Transfer — a push

A **credit transfer** is the salary-and-invoice workhorse: the payer's bank initiates and *pushes* funds to the payee. No mandate is needed — you're sending your own money. In the European SEPA system this is the *SEPA Credit Transfer* (SCT), settling at T+1, and it corresponds to the ISO 20022 **pacs.008** interbank message. The worked example in Chapter 11 *was* a credit transfer; there is nothing more to it mechanically.

(The mention of **pacs.008** points at a real-world detail worth noting: actual interbank payments are carried as standardized ISO 20022 XML messages with codes like **pacs.008** for a credit transfer and **pacs.003** for a direct debit. A learning model can stand in a simple payment record for those messages and just *name* the message each scheme corresponds to, without parsing real ISO 20022.)

SEPA Direct Debit — a pull, with a mandate

A **direct debit** runs the other way. The payee — a utility, a gym, a streaming service — *pulls* funds from the payer's account. Since someone reaching into your account is obviously dangerous, a direct debit requires a **mandate**: a standing authorization you sign in advance, naming the specific creditor allowed to collect and often capping the amount.

Before any collection, the scheme checks that mandate: that it exists, is still active (not revoked), matches both the payer and the named payee, and stays within its amount limit. Fail any check and the payment is rejected outright — with specific reasons like “mandate required,” “mandate revoked,” or “amount exceeds the mandate.” In SEPA this is the *SEPA Direct Debit* (SDD), settling at T+2, mapping to the ISO 20022 **pacs.003** message.

Here's the payoff of the shared-machinery design: once the mandate checks pass, the *postings for a direct debit are identical to a credit transfer* — debtor → creditor, through clearing suspense and central-bank reserves, exactly as in Chapter 11. The money flows the same way; only the rules about *who may initiate* and *what authorization is required* differ. That's the whole reason schemes can be an abstraction over one posting engine rather than separate systems.

Returns: unwinding a settled payment

Direct debits introduce a wrinkle the credit transfer didn't have: they can be **returned**. If the payer disputes a collection or turns out to have lacked funds, a *return* (in SEPA, an "R-transaction") reverses the flow even after settlement. It posts compensating transactions that move the funds back from creditor to debtor across the central bank, fully unwinding the original and restoring both customers' balances — the cross-bank cousin of the reversal from Chapter 5. Where an ordinary reversal undoes a posting in one ledger, a return has to undo a flow that crossed several ledgers and the central bank, but the principle is the same: you don't erase history, you post a new, balancing transaction that cancels it.

SEPA moves money from debtor to creditor by agreement between two parties — a payer and a payee, governed by a mandate when the payee pulls. Cards introduce a different *shape* entirely: a four-party network (cardholder, merchant, issuing bank, acquiring bank) and a two-step life of authorize-then-settle that spans Chapter 13.

Chapter 13 — Card Transactions

A card payment is another payment scheme, no more magical than the credit transfer we traced in Chapter 11. But it has a different *shape* than SEPA. When you tap or swipe, the first thing that happens is not a posting at all — it is an **authorization**, and an authorization is exactly the **hold** of Chapter 7: a real-time reservation against your available balance that moves no money and leaves your book balance untouched. The money moves later, in a separate step. Two structural differences set cards apart from the SEPA flows, and the whole chapter is about them: there are more parties involved, and there is a deliberate gap between the moment a payment is *authorized* and the moment it is actually *settled*.

The four-party model

A SEPA payment is, at heart, a two-party affair: a debtor's bank and a creditor's bank, money pushed or pulled from one to the other. A card payment routes the same debtor → creditor flow through **four** parties plus a network that connects them.

- The **cardholder** — you — who owns the card and the deposit account behind it.
- The **issuer** — the cardholder's bank, which issued the card. The issuer is the one that authorizes a purchase against your account and ultimately *debts* you.
- The **merchant** — the shop or website being paid.
- The **acquirer** — the merchant's bank, which "acquires" the transaction on the merchant's behalf and ultimately *credits* the merchant.
- The **card network** (or **scheme**) — Visa, Mastercard, and their peers — which sits between the issuer and the acquirer, carrying messages each way and computing what the banks owe one another.

Lining up who faces whom makes the shape clear. The cardholder faces the merchant (the purchase). The issuer faces the acquirer through the network (the interbank leg). And each bank faces its own customer: the issuer debits the cardholder, the acquirer credits the merchant. The network is the hub the two banks meet at — it never holds the money, it just connects the issuer's side to the acquirer's side and, at the end of the day, tells each bank its net position, exactly the way the clearing cycle of Chapter 11 did. SEPA had the two banks meeting at a central bank; cards add a scheme in the middle that routes the messages and runs the netting, but the underlying motion is still one customer's balance falling and another's rising.

Authorization → presentment

The defining feature of a card payment is that it happens in two messages, separated in time. This is the **dual-message** flow.

The first message is the **authorization**. At the point of sale, the terminal asks the issuer, in real time: *will you stand behind this amount?* The issuer checks the cardholder's available balance and account state, and if all is well, it approves — and places a **hold**. This is the authorization–capture pattern of Chapter 7, arriving exactly on cue. No money moves. The cardholder's **book balance is unchanged**; only the **available balance** drops by the authorized amount, so the funds can't be spent twice while the purchase

is in flight. The approval travels back to the terminal, the cardholder walks out with the goods, and yet nothing has been posted to any ledger at all.

The second message comes later — often that night, sometimes a day or two on — when the merchant, through its acquirer, submits the **presentment** (also called the *clearing record*): the formal claim that says *this sale really happened, here is the final amount, please pay*. It is on presentment that the issuer turns the hold into a real posting. The hold is **captured**, and the captured amount becomes the **debtor leg** of an actual transaction — the cardholder's deposit liability is debited for the settled amount, just as a captured hold became a real posting in Chapter 7:

```
Issuer: Debit Cardholder (Liability)      4500 // available was already reduced; now the book balance falls
        Credit Clearing Suspense (Liability) 4500 // the issuer now owes the network, in flight
```

From here the payment is an ordinary in-flight debtor leg, parked in clearing suspense, waiting for the cycle to settle — which is the next section's business.

Not every card transaction is split in two. The **single-message** variant fuses authorization and clearing into one step, with no separate presentment to follow. The familiar case is a **PIN debit** purchase or an **ATM** withdrawal: you enter your PIN, the issuer approves *and* commits the debit in the same breath, and the money is gone immediately rather than held and captured later. There is no gap to manage because authorization and clearing are the same message. The mental model collapses to a single posting at the moment of approval — closer in feel to an instant payment than to the held-then-captured purchase above. Most card purchases are dual-message; the single-message path is the exception that proves how the two halves of the flow are normally distinct steps that merely happen, here, to coincide.

Authorized $\neq$ final amount

The two-message gap creates a possibility SEPA never had: the amount *authorized* and the amount eventually *captured* can differ. This is not a bug — it is the whole reason the held amount is provisional. A hold reduces the *available* balance, not the *book* balance (Chapter 7), so the issuer can reserve an estimate now and settle the truth later, and the books are never wrong in the meantime because nothing was posted.

Three everyday cases show the gap at work.

- **The gas-pump pre-authorization.** This is the canonical example from Chapter 7. The pump can't know how much fuel you'll buy, so it authorizes a fixed estimate — say a \$100 hold — before a drop is pumped. When you finish at \$45, the presentment captures \$45, and the remaining \$55 of reservation is released. The hold was a placeholder; only the capture was real.
- **The restaurant tip.** A waiter brings the check, you authorize the meal, and *then* you write in a tip on the slip. The captured amount comes in *higher* than what was authorized — the tip is added after the authorization — and the issuer honors it. The authorization established that you were good for roughly this much; the final amount adjusted within an expected tolerance.
- **Partial and incremental captures.** A single authorization need not be settled all at once. A merchant shipping an order in two boxes may take a **partial capture** for the part that shipped, leaving the rest of the hold to be captured or released later. A hotel that authorized a room for three nights and then watched you order room service may add an **incremental** authorization, raising the hold before the final capture. The reservation flexes; the ledger only records the amounts actually settled.

In every case the same discipline from Chapter 7 holds: the reservation lives off the ledger and can be adjusted, expired, or released freely, and the ledger learns the final figure exactly once, at capture. The account *states* of Chapter 8 govern this path too. An authorization is a debit-side request, so a **frozen** account blocks it — the issuer refuses the authorization outright, no hold is placed, and the purchase is declined at the terminal, the same way a frozen account blocks any new debit.

Reusing the rails

Here is the quiet payoff. Once a hold is captured, a card payment stops being special. The captured amount is now an ordinary **debtor** → **creditor** flow — the cardholder's issuer owes, the merchant's acquirer is owed — and from that point it nets and settles through the exact machinery of Chapter 11. Card networks compute each bank's net position over a cycle and settle the net, just as SEPA does; the issuer's clearing suspense drains into a central-bank reserve move, the acquirer's fills the merchant's account, and the books reconcile by the mirror-image rule we have leaned on throughout. Clearing and settlement (Chapters 9–11) need no new mechanism for cards; the authorization was the only genuinely new idea, and we already had it as a hold.

This is also why a card scheme is a small extension rather than a new system. Our reference library already models authorization as holds and already has capture, so a full card scheme would extend from there — adding the four-party routing and the presentment step on top of machinery that, underneath, is unchanged.

And with that, the journey of Part IV is complete. We have followed money from a single balanced entry in one bank's book all the way to final settlement across an interbank network, and through every scheme that moves it — a credit transfer pushed, a direct debit pulled, an instant payment settled gross, and now a card payment authorized, captured, cleared, and settled. The motion is finished. The last part of the book steps back from *movement* to *records*: how a bank snapshots its balances at the close of each day, keeps an unbreakable audit trail of everything that ever happened, and distills it all into the statements that are the only part of this machinery a customer ever sees.

Chapter 14 — Snapshots, Audit Trails, and Statements

We've followed money from a single balanced entry all the way to final settlement across an interbank network. What remains is the bank's memory: how it captures balances at the close of each day, how it keeps an unbreakable record of everything that ever happened, and how all of it is distilled into the one artifact the customer actually sees — the monthly statement. This closing chapter is about turning a living ledger into trustworthy reports.

End-of-day snapshots

A bank computes balances constantly, but at the close of each business day it does something special: it captures a **snapshot** of every account's balance and stores it. A snapshot is a frozen record of where things stood at end of day — and it earns its keep in several ways at once:

- **Interest accrual.** Daily interest is calculated on the end-of-day balance. For a savings account earning 4% a year, the day's interest on a \$10,000 balance is $\$10,000 \times 0.04 / 365 = \1.10 . Without a captured end-of-day figure, there's nothing to accrue against.
- **Statement generation.** Monthly statements show the balance at the end of each day, alongside activity and opening/closing balances. The snapshots are the raw material.
- **Regulatory reporting.** Banks must report their positions to regulators, and end-of-day balances are the standard reporting granularity.
- **Performance.** Rather than replaying every transaction since an account was opened to answer "what's the balance?", the system can start from the most recent snapshot and replay only the transactions since. The snapshot is a checkpoint.

Which balance does a snapshot record? The *value-date* balance from Chapter 7 — the economically real one — because that's the balance interest and regulation care about. This is the booking-vs-value distinction (Chapter 6) determining, once again, which number is the one that counts. In a layered system, snapshots are naturally a job for the deposit layer that tracks the three-part balance, not the pure ledger that merely computes book balances on demand.

The audit trail

Underneath every snapshot and every report lies the bank's most fundamental record: the **audit trail**. It is an immutable, append-only log of every change the system has ever made. Nothing is ever deleted from it. Every account opening, every posted transaction, every hold placed or released, every reversal, every snapshot — each is recorded with:

- a unique event identifier,
- a timestamp,

- the type of event,
- the entity it affected, and
- the full details of what happened.

This is the same append-only philosophy we met with ledger immutability in Chapter 5, generalized from transactions to *every* mutation in the system. And it buys the bank four things it cannot do without:

- **Regulatory compliance** — banks are legally required to keep complete records.
- **Forensic investigation** — when something looks wrong, the trail shows exactly what happened and in what order.
- **Debugging and incident response** — the same visibility serves engineers.
- **Independent verification** — because every mutation is logged, balances can be *recomputed from scratch* by replaying the events, and checked against what the system currently reports. If they ever disagree, you have caught a bug or a corruption. It is the trial balance of Chapter 2, taken to its logical end: not just “do the debits equal the credits?” but “does the entire current state follow from the recorded history?”

An immutable log that lets you rebuild the present from the past is the strongest form of the trustworthiness this whole book has been building toward.

Statements: the customer's view

Finally we reach the one report a customer actually reads: the **bank statement**, a periodic (usually monthly) summary of activity and balances on their account. The statement is where the abstractions of this book surface in plain sight — and where the two clocks of Chapter 6 produce their most visible, most-complained-about effect.

A statement shows three things:

- **A transaction listing** — every transaction with a *booking date* in the period, in chronological order. This is the activity the customer recognizes: “I paid this on the 3rd, I got paid on the 15th.” Booking date is used here because it's the date the customer associates with *doing* the transaction.
- **Opening and closing balances** — computed using the *value date*. The opening balance is the end-of-day value-date balance from the last day of the prior period; the closing balance reflects every transaction whose value date falls within this period.
- **Daily balances** — the value-date balance at the end of each day, used for interest and regulatory purposes. (These are exactly the snapshots from the start of this chapter.)

Why the statement may not seem to add up

Here is the subtle finish. Because the transaction *listing* uses booking date while the *balances* use value date, the listed transactions may not perfectly reconcile with the balance change shown — and this is correct behavior, not a bug:

- A transaction **booked February 25** with a **value date of March 1** appears in February's listing (you did it in February) but does *not* affect February's closing balance (it isn't economically real until March).
- A transaction **booked January 31** with a **value date of February 1** does *not* appear in February's listing (you did it in January) but *does* affect February's opening balance (it became real on February 1).

So a careful customer adding up February's listed transactions may not arrive at February's balance change — because some listed activity hasn't taken value yet, and some value that landed in February was for activity listed in January. This is why most retail statements print *both* dates whenever they differ: it's the bank showing its work, so the apparent mismatch resolves into sense. The closing balance and the daily

balances come from the value-date snapshots precisely because those are the figures that drive interest and that regulators expect — the same reason snapshots record the value-date balance in the first place.

Where we've arrived

We began with a single rule — every transaction has equal debits and credits, so money is only ever moved, never created — and a single reframing — that your deposit is the bank's promise to you. From those two seeds grew everything since: the five account types and their normal balances; the integer minor units that keep every cent exact; the append-only transactions that are corrected only by reversal; the two clocks of booking and value date; the three balances and the holds that bridge them; the lifecycle that governs an account from opening to closure; and finally the interbank machinery that moves value between banks through reserves at a central bank, clearing the net and settling with finality.

What ties it all together is a single ambition that has run through every chapter: to build a record of money so disciplined that it is *provably* correct, *completely* auditable, and *trustworthy* down to the last cent. That discipline — not any particular piece of software — is what a core banking system really is.